

M19 / PROTECTED ANSWERS

M19 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	4
DE19-01 Truthful one-page Data Engineer resume	4
DE19-02 GitHub and project portfolio final polish	4
DE19-03 LinkedIn/professional profile without overclaiming	5
DE19-04 Degree-aware job targeting: apply/skip rules	5
DE19-05 SQL interview drills	5
DE19-06 Python interview drills	6
DE19-07 Data-engineering concepts and troubleshooting drills	6
DE19-08 Project walkthrough interview	7
DE19-09 Architecture/system-design defense for junior roles	7
DE19-10 Behavioral answers and career-transition story	7
DE19-11 Technical English: explain code, failures and decisions clearly	8
DE19-12 Application tracker, feedback loop and weekly repair cycle	8
DE19-13 Timed coding test: arrays, strings and hash-map problems	8
DE19-14 Combined SQL + Python interview simulation	9
Worked SQL interview answers	9
SQL 1 - reasoning and result	9
SQL 2 - reasoning and result	10
SQL 3 - reasoning and result	10
SQL 4 - reasoning and result	11
SQL 5 - reasoning and result	11

SQL 6 - reasoning and result	11
Worked Python interview answers	12
A worked junior system-design answer	13
A truthful project walkthrough	13

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open the matching review only after saving a genuine attempt. Each review gives one defensible answer shape, checks and a changed retry.

DE19-01 | Truthful one-page Data Engineer resume

One acceptable solved shape: A defensible resume maps every technical bullet to evidence. For an entry-level transition candidate, projects belong under Selected Projects or Technical Projects - not disguised as employment. A project bullet should name the engineering boundary and proof, such as immutable raw landing + quarantine + reconciliation + replay-safe state. "Improved performance 70%" is rejected unless you actually measured a comparable baseline and saved the evidence.

Validation: One page at normal font size; no tiny compressed text. | No "experience" language for a course project unless it is explicitly labelled project experience. | Every named technology appears in at least one repository or lab.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Take one project bullet and remove the technology names. Explain the engineering problem and validation first; then add only the tools that materially solved it.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-02 | GitHub and project portfolio final polish

One acceptable solved shape: A strong profile pins only projects you can defend. The first README screen should state the business problem, architecture/data flow, grain/state, quick-start, expected controls, failure/recovery and limitations. Generated outputs belong only when they help reviewers; caches, secrets and machine-specific paths do not.

Validation: 3-5 strongest projects surfaced, not every practice folder. | Each project has problem -> architecture -> run -> validate -> failure/recovery -> limitations. | No credentials, tokens, personal secrets or machine-specific absolute paths.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Give the README to someone who has not seen the course. Ask them to state the grain, input, output and proof in two minutes. Repair whatever they cannot find.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-03 | LinkedIn/professional profile without overclaiming

One acceptable solved shape: Truthful headline example: “Data Engineering Portfolio | Python • SQL • Spark • Airflow • Delta/Fabric | Replay-safe pipelines & analytics engineering.” It signals the target without claiming current employment. The About section should link deliberate learning to two or three concrete projects and one engineering principle you can demonstrate.

Validation: Headline states direction/evidence without inventing current employment. | About section explains transition, strongest engineering evidence and target role in plain language. | Skills list is smaller than your course index; it contains tools you can defend.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Remove all adjectives such as “expert”, “advanced”, “scalable”, “enterprise”. Add back only terms you can support with a concrete design or result.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-04 | Degree-aware job targeting: apply/skip rules

One acceptable solved shape: The apply/skip decision begins with exact employer language. “CS degree preferred” is not the same as “B.Tech CSE mandatory.” “Degree or equivalent practical experience” explicitly creates an evidence route. Hard gates are respected; preferences become priority weights; skill gaps become repair tasks. Never rewrite the JD in your head.

Validation: Decision names the exact hard gate or equivalent clause. | Core skill match is supported by project evidence, not course completion alone. | Logistics such as location/interview/shift are tracked separately.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Re-score one borderline job after removing all “preferred” requirements. Did your decision change for the right reason?

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-05 | SQL interview drills

One acceptable solved shape: For SQL, a worked answer starts with output grain and control totals. For latest-paid-order questions, RANK can intentionally surface timestamp ties; ROW_NUMBER requires a deterministic secondary rule. Join fan-out is checked with row/key counts before accepting aggregates. “Query ran” is not a correctness test.

Validation: State grain before each query. | Check row count/key uniqueness and one aggregate control. | Explain why the join/ranking does not duplicate business entities.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary

rather than adding more buzzwords.

Changed retry: Solve one drill a second way (CTE vs subquery, GROUP BY vs window) and prove equivalent business results.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-06 | Python interview drills

One acceptable solved shape: Python review: first duplicate uses a set; counts use a frequency map; first non-repeating character uses counts plus original-order scan; latest-version dedupe must reject conflicting equal max versions rather than silently choose by input order. Complexity and edge cases are part of the answer.

Validation: Functions handle empty input, duplicates and missing values specified in the prompt. | Tests pass because behavior is correct, not because tests were weakened. | You can explain the chosen data structure in one sentence.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Rewrite one solved function with a deliberately worse data structure, compare complexity, then restore the better version.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-07 | Data-engineering concepts and troubleshooting drills

One acceptable solved shape: Incident answers protect evidence first. A missing-row + advanced-checkpoint incident is a state/commit problem until evidence proves otherwise. Freeze progress, compare IDs/counts, separate bad-data quarantine from infrastructure failure, repair ownership, replay only unsafe work, then reconcile.

Validation: No destructive recovery before source/sink/state evidence is saved. | Checkpoint/offset advancement is tied to durable validated work. | Bad data and infrastructure failure are separated.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Take one incident and assume your first hypothesis is wrong. Name the evidence that would prove it wrong and the next hypothesis.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-08 | Project walkthrough interview

One acceptable solved shape: Project defense should spend little time listing tools. A strong answer says what one row/event represents, what was durable, why a design was chosen, what failed, how it recovered, and how correctness was independently checked. End with one honest limitation rather than pretending production scale.

Validation: Problem and grain appear before tool list. | At least one decision includes an alternative and trade-off. | Failure/recovery is specific and evidence-backed.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Let a reviewer interrupt after any sentence with “why?” or “how do you know?”. Repair answers that collapse into “because the tutorial said so”.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-09 | Architecture/system-design defense for junior roles

One acceptable solved shape: System-design defense follows requirements -> scale -> flow -> state -> quality -> failure -> security -> cost -> trade-offs. Tool names come after constraints. Changing daily latency to 30 seconds should not automatically replace every component; only the latency-sensitive path must change.

Validation: Architecture choices follow requirements rather than tool preference. | Durable raw/replay path is explicit. | You state at least one failure domain and recovery.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Change latency from daily to 30 seconds. Redesign only the parts that truly need to change and defend why the rest stays.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-10 | Behavioral answers and career-transition story

One acceptable solved shape: A STAR story is specific. Use “I” for your action and keep Result factual. A course project bug is valid if described honestly as project evidence. A transition story should connect previous foundation, deliberate technical build, proof and target - without apologizing for degree background or inventing work experience.

Validation: Stories are specific enough that dates/project context make sense. | Action uses “I” for your contribution. | Result is factual even if modest.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Cut one 3-minute story to 90 seconds without deleting the decision or evidence.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-11 | Technical English: explain code, failures and decisions clearly

One acceptable solved shape: Clear technical English uses concrete nouns and causal links: “The task sensor waited because the fixture path did not match the DAG path” is stronger than “it failed and I fixed it.” State uncertainty as a hypothesis, then name evidence and next check.

Validation: Sentence names the object (“checkpoint”, “sink write”, “order grain”) rather than “it/thing”. | Uncertainty is explicit; you do not present hypotheses as facts. | Answer has beginning/problem, reasoning/action, result/next step.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Explain the same incident to (a) an engineer and (b) a nontechnical manager. Keep the facts constant; change only depth and vocabulary.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-12 | Application tracker, feedback loop and weekly repair cycle

One acceptable solved shape: Use funnel evidence. Zero screens after well-targeted applications points first to targeting/resume/profile. Screens but technical failures point to SQL/Python/project defense. Technical passes but final-round failures may point to architecture/behavioral/communication. Change one variable per weekly repair cycle.

Validation: Tracker has role URL/company/date/stage/hard-gate decision and next action. | No fake recruiter/interview status. | Weekly repair is linked to a funnel signal, not random course hopping.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Run the weekly review on the supplied sample tracker, then create a second scenario where screens are high but coding-round conversion is low. Choose a different repair.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-13 | Timed coding test: arrays, strings and hash-map problems

One acceptable solved shape: Master coding patterns untimed first. For first non-repeating character, frequency map + second scan is $O(n)$. Optional timing comes only after correct tests. If a timed run fails, record whether the bottleneck was problem interpretation, data-structure choice, coding or debugging.

Validation: All supplied cases pass without modifying expected results. | You can explain $O(n)/O(n^2)$ choices. | Empty/duplicate/no-solution cases are explicit.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Change a list problem into a streaming iterator where you cannot sort or rewind. Redesign the state.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

DE19-14 | Combined SQL + Python interview simulation

One acceptable solved shape: Combined simulation base controls are paid orders=4 and total paid amount=500.00; customer totals C1=200 and C2=300. Python derived flags must preserve row count and total. A timestamp tie can change “latest order” identity but must not change aggregate paid totals; state a deterministic tiebreaker or surface ambiguity.

Validation: SQL intermediate grain is explicit and unique. | Aggregate totals reconcile to source controls. | Python never changes business totals while adding derived flags.

Common wrong answer: a polished statement with no artifact/control, a hidden ambiguity, or a claim that cannot survive one follow-up question. Repair the smallest unsupported boundary rather than adding more buzzwords.

Changed retry: Add one new paid order tied at the existing maximum timestamp and prove your SQL/Python contract does not skip or double count it.

Teach-back: close this review and explain the decision, evidence and limitation in plain language. If you need this paragraph open, the lesson is not yet cold-ready.

Worked SQL interview answers

These queries were executed against practice/sql/interview_fixture.sqlite. This classroom SQLite fixture uses small exact-looking numeric examples; production money should use integer minor units or an exact decimal database type. Save your first attempt before using these answers.

SQL 1 - reasoning and result

Filter orders to PAID before grouping by the customer region. EAST is 360.00 and WEST is 260.00, which reconcile to 620.00. Grouping by customer would answer a different question. An INNER JOIN is appropriate here because only regions with paid orders are requested.

```
SELECT c.region, ROUND(SUM(o.amount), 2) paid_revenue
FROM orders o
JOIN customers c USING(customer_id)
WHERE o.status='PAID'
GROUP BY c.region
ORDER BY c.region;
```

region	paid_revenue
EAST	360.0

region	paid_revenue
WEST	260.0

SQL 2 - reasoning and result

NOT EXISTS asks whether a matching paid order exists, without multiplying the outer customer row. C4 is the only result. Testing status != PAID would be wrong because a customer can have both paid and unpaid orders.

```
SELECT c.customer_id
FROM customers c
WHERE NOT EXISTS
  (SELECT 1
   FROM orders o
   WHERE o.customer_id=c.customer_id
        AND o.status='PAID')
ORDER BY c.customer_id;
```

customer_id
C4

SQL 3 - reasoning and result

Filter to paid orders before ranking within each customer. RANK deliberately exposes ties: C2 has O3 and O4 at the same latest timestamp, so four rows are correct for the stated tie policy. To require exactly one row, use ROW_NUMBER with an explicit order_id tie-break and explain that choice.

```
WITH p AS
  (SELECT *, RANK() OVER(PARTITION BY customer_id
                        ORDER BY updated_at DESC) r
   FROM orders
   WHERE status='PAID')
SELECT *
FROM p
WHERE r=1
ORDER BY customer_id,order_id;
```

order_id	customer_id	updated_at	status	amount	r
O2	C1	2026-09-02T10:00:00Z	PAID	80.0	1
O3	C2	2026-09-02T09:00:00Z	PAID	220.0	1
O4	C2	2026-09-02T09:00:00Z	PAID	40.0	1
O6	C3	2026-09-04T11:00:00Z	PAID	160.0	1

SQL 4 - reasoning and result

Order by timestamp and the unique order_id to make ties deterministic. The running totals are 120, 340, 380, 460 and 620. The final running total must match the independently grouped paid total. A nonunique ordering could leave the intermediate path ambiguous.

```
SELECT order_id,updated_at,amount,SUM(amount) OVER(
                                ORDER BY updated_at,
                                order_id) running_paid
FROM orders
WHERE status='PAID'
ORDER BY updated_at,order_id;
```

order_id	updated_at	amount	running_paid
O1	2026-09-01T10:00:00Z	120.0	120.0
O3	2026-09-02T09:00:00Z	220.0	340.0
O4	2026-09-02T09:00:00Z	40.0	380.0
O2	2026-09-02T10:00:00Z	80.0	460.0
O6	2026-09-04T11:00:00Z	160.0	620.0

SQL 5 - reasoning and result

Aggregate per customer first, then rank those aggregates. C2 is top with 260.00. Ranking individual orders would find the biggest order, which is a different business question. RANK also retains all customers in a tied first place.

```
WITH x AS
  (SELECT customer_id,SUM(amount) amt
   FROM orders
   WHERE status='PAID'
   GROUP BY customer_id),
  r AS
  (SELECT *,RANK() OVER(
                                ORDER BY amt DESC) pos
   FROM x)
SELECT *
FROM r
WHERE pos=1;
```

customer_id	amt	pos
C2	260.0	1

SQL 6 - reasoning and result

Put the PAID condition in the LEFT JOIN ON clause to preserve customers with zero paid orders. COUNT(order_id) ignores the unmatched null; COUNT(*) would incorrectly give C4 a count of one. The four rows are C1 2/200, C2 2/260, C3 1/160, C4 0/0. Their amounts reconcile to 620.00.

```
SELECT c.customer_id,COUNT(o.order_id) paid_orders,
       COALESCE(SUM(o.amount), 0) paid_amount
```

```
FROM customers c
LEFT JOIN orders o ON o.customer_id=c.customer_id
AND o.status='PAID'
GROUP BY c.customer_id
ORDER BY c.customer_id;
```

customer_id	paid_orders	paid_amount
C1	2	200.0
C2	2	260.0
C3	1	160.0
C4	0	0

Worked Python interview answers

`first_duplicate([A, B, A, C])` returns A because the second A is the first item already in the seen set. `counts_by_status` counts each supplied row once. `first_non_repeating("swiss")` returns w: s occurs three times, while w is the first character whose total count is one. These functions are linear in the number of input items under normal dictionary/set assumptions. For latest-version deduplication, do two passes: discover each ID's maximum version, then retain and compare only rows at that version. Equal maximum versions with different content raise `ValueError`; older disagreements do not change this interview answer. Reversing input must not change the result. `reconcile_ids` reports missing source IDs and unexpected target IDs; because it uses sets, it does not detect multiplicity errors. Use counts separately when duplicate frequency matters.

```
from collections import Counter
```

```
def first_duplicate(ids):
    s = set()
    for x in ids:
        if x in s:
            return x
        s.add(x)
    return None
```

```
def counts_by_status(rows):
    return dict(Counter(r["status"] for r in rows))
```

```
def first_non_repeating(text):
    c = Counter(text)
    return next((ch for ch in text if c[ch] == 1), None)
```

```
def dedupe_keep_latest(rows):
    """Keep highest version per id; reject conflicting content at that maximum.
    Older conflicting versions are irrelevant to this particular interview contract.
    """
    rows = list(rows)
    maximum = {}
    for row in rows:
        key = row["id"]
        maximum[key] = max(maximum.get(key, row["version"]), row["version"])
```

```
result = {}
for row in rows:
    key = row["id"]
    if row["version"] != maximum[key]:
        continue
    if key in result and result[key] != row:
        raise ValueError("conflicting latest version")
    result[key] = dict(row)
return result

def reconcile_ids(source_ids, target_ids):
    return {
        "missing": set(source_ids) - set(target_ids),
        "unexpected": set(target_ids) - set(source_ids),
    }
```

A worked junior system-design answer

For a small daily orders report, I would begin with scheduled batch ingestion into immutable raw storage, validate types and relationships, merge current orders using order_id and a source version, and publish a checked SQL summary. I would record a logical batch date and advance progress after successful output validation. This avoids forcing a streaming system onto a daily business requirement. My main failure case is a process stopping after the sink write but before progress commits. The repeated run must recognize the same source identity and make the same business change at most once. I would prove this with a deliberately failed run and a replay, comparing order keys and paid revenue. I would keep raw access restricted, avoid secrets in code and expose only the fields analysts need. Before scaling, I would measure arrival volume, processing duration, skew and the report's freshness requirement. If the interviewer changes the requirement to second-level freshness, I would reconsider source CDC, partitioned event delivery and streaming state. I would explicitly discuss out-of-order events, retention, replay, sink identity and operational cost. That is a requirement change, not evidence that the original daily batch design was wrong.

A truthful project walkthrough

My supplied capstone joins orders to captured payments while keeping unpaid orders visible. The fixture has 12 orders worth 1380.00, but captured payments total 1240.00 because OR009 was declined. I aggregate captures at order grain before the join and check both event identity and the business rule against duplicate captures. I deliberately inject a duplicate and prove publication stops. In a live interview, replace these sentences with your own executed evidence; do not claim a native deployment you only designed.